Nyaya Saathi: a grounded AI legal helper for India

Kuku AI Generalist Assignment, Problem 2. Prototype: Streamlit app + CLI (repo link in submission form).

1. Why this scope

Most people give up on everyday disputes. I picked **three high-frequency problems** where the path is fairly standard across India: **unpaid refunds/defective goods** (consumer), **rental deposit not returned**, and **unpaid salary**, plus **cheque bounce** because its deadlines are strict and easy to miss. Depth beats breadth: the bot does three jobs: (1) **build the case** (what / when / amount / proof held / proof missing as a "case card"), (2) **explain options** in the user's language, (3) **draft a notice**, and (4) **practise vs the other side** (role-play + coach note).

2. Assumptions

Users type in Hindi, Hinglish, Bengali or English (the model replies in the same language). Users are not lawyers and the answer is **legal information, not advice**. State laws differ, so where a state-specific rule matters the bot says so rather than guessing. Voice is out of scope for 3 hours; the planned route is browser speech-to-text in front of the same text pipeline. KB facts reflect my reading of public sources as of Nov 2026 and need lawyer review before real use.

3. How it avoids inventing law (the core design)

Layer	What it does	AI or code?
Route	Keyword router picks topic (consumer / tenancy / employment / cheque / general) and flags **escalation words** (FIR, arrest, custody, dowry...)	Code
Ground	Model only sees a **hand-curated facts file** (kb/laws.json, 9 entries, each with source + verify link). Prompt says: law claims only from this block, each tagged like [CPA-LIMIT]	Code + prompt
Talk	Model asks up to 3 questions if facts are missing, then explains in plain language; returns JSON (reply, case card, kb ids used, needs_lawyer)	LLM
Check	**Regex guardrail**: any Section / Act / case-looking string or tag not in the KB is flagged; the reply gets a visible warning and a lawyer nudge (fail closed)	Code
Escalate	Criminal, family, property-title, court-notice topics get a fixed message pointing to a real lawyer / DLSA helpline 15100	Code

Why a small KB instead of "ask the model"? Models confidently invent sections and cases, and also quote repealed law (e.g. IPC after BNS replaced it on 1 July 2024; the KB has a renumbering entry for this). A short verified list is auditable by a lawyer in an hour. Unsure answers become "check with a lawyer" instead of a guess.

4. Quality checking: what I actually ran

Check	Result
Guardrail unit test (offline, 3 planted hallucinations: fake case, fake Act 2031, IPC 420, unknown tag)	Flagged all of them; passed a correct reply with a real Section 69 citation. File: raw_outputs/guardrail_unit_test.json
Router test on 6 sample sentences	6/6 routed correctly; "FIR and arrest" escalated. First version wrongly escalated on the word "will" (substring match), so I switched to word-boundary matching and removed "will" from the trigger list.
7 end-to-end test cases (tests/cases.json): Hinglish deposit, refund, Bengali salary, cheque, arrest, an **uncovered** land question, a time-barred claim	**Not yet run by me on a live model** (needs your API key). run_tests.py writes unedited outputs to raw_outputs/ with pass/fail on must-say and must-not-say strings. Please treat the end-to-end quality as unproven until those files exist.

5. Where it fails (known, honest)

(a) The guardrail is a regex: it catches invented sections/Acts/cases, but **cannot catch a wrong number inside correct-looking prose** (e.g. the wrong deadline), which is why every fact is tagged and verify links are shown. (b) Keyword routing misses messages that use unusual words or mix topics. (c) The KB is tiny and not state-aware: deposit rules depend on the state and on the agreement. (d) The Code on Wages vs Payment of Wages Act applicability is left as "verify", deliberately. (e) Drafted notices contain placeholders and always carry "get a lawyer to review". (f) No voice, no persistent storage; personal data is not saved.

6. Tools used, and where AI got it wrong

I used Claude to draft the code, the KB wording and this document; decisions on scope and guardrails are mine. **One place it went wrong:** the first draft of the escalation rule used simple substring matching and treated the everyday word "will" as a trigger for inheritance/wills, escalating ordinary questions. I found it by testing the router and fixed it with word boundaries.

7. Scale and cost

Adding a new law = adding one JSON entry (no code change); a lawyer or law student can maintain it. The model is swappable by 3 env vars (Anthropic or any OpenAI-compatible endpoint, including Gemini/Groq free tiers), so stronger models can be plugged in. One chat turn is a single call with a small prompt (about 1.5-2k tokens), so a small, fast model is enough; the expensive step (human review of the KB) happens once per law, not per user. At volume I would add: state-wise KB tags, retrieval by embeddings once the KB passes ~100 entries, and logging of guardrail flags to find gaps.

8. How I would know it works with real users

Lawyer review of 50 sampled answers (wrong-law rate target: 0 invented citations); % of conversations ending with the user holding a case card or notice they act on; escalation precision (are flagged cases really lawyer-worthy?); and guardrail flag rate trending down as the KB grows.